

Classification Overview



Topics

- Naive Bayes
- Logistic Regressions
- Decision Trees
- Random Forest
- scikit-learn

Naive Bayes

The diagram illustrates the Naive Bayes formula, $P(H|E) = \frac{P(E|H) * P(H)}{P(E)}$, with arrows pointing from descriptive labels to the corresponding parts of the equation. The labels are: 'Likelihood of the Evidence given that the Hypothesis is True' (yellow) pointing to $P(E|H)$; 'Prior Probability of the Hypothesis' (red) pointing to $P(H)$; 'Posterior Probability of the Hypothesis given that the Evidence is True' (blue) pointing to $P(H|E)$; and 'Prior Probability that the evidence is True' (green) pointing to $P(E)$.

Likelihood of the Evidence given that the Hypothesis is True

Prior Probability of the Hypothesis

$$P(H|E) = \frac{P(E|H) * P(H)}{P(E)}$$

Posterior Probability of the Hypothesis given that the Evidence is True

Prior Probability that the evidence is True

Naive Bayes (Example)



HAM

Total emails: 72

Prob. Ham
= 72/100
= 0.72

Prob. Spam
= 28/100
= 0.28

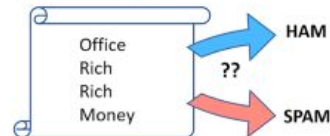


SPAM

Total emails : 28

HAM Total Words 268			
Word	Count	Count New	Prob of Word if mail is Ham
Friend	86 +1	87	0.324627
Rich	3 +1	4	0.014925
Money	9 +1	10	0.037313
Beach	90 +1	91	0.339552
Office	75 +1	76	0.283582

SPAM Total Words 248			
Word	Count	Count New	Prob of Word if mail is Spam
Friend	59 +1	60	0.241935
Rich	63 +1	64	0.258065
Money	97 +1	98	0.395161
Beach	24 +1	25	0.100806
Office	0 +1	1	0.004032



Probability new email is Ham

$$P(H) \times P(\text{Office}|H) \times P(\text{Rich}|H) \times P(\text{Rich}|H) \times P(\text{Money}|H)$$

$$0.72 \times 0.28 \times 0.01 \times 0.01 \times 0.04 = \mathbf{0.0000017}$$

Probability new email is Spam

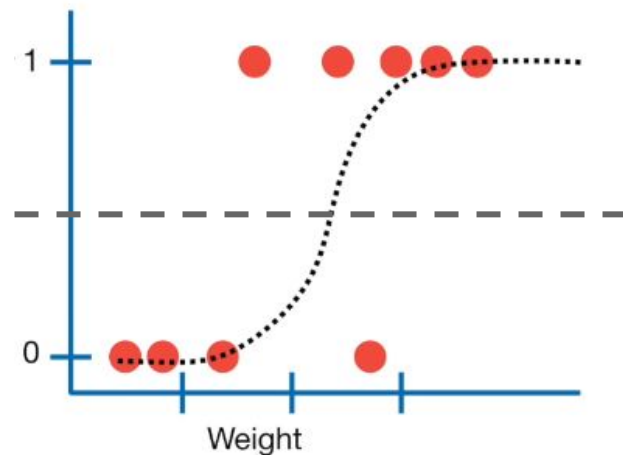
$$P(S) \times P(\text{Office}|S) \times P(\text{Rich}|S) \times P(\text{Rich}|S) \times P(\text{Money}|S)$$

$$0.28 \times 0.004 \times 0.25 \times 0.25 \times 0.39 = \mathbf{0.000029}$$

SPAM!!!

Logistic Regression

- Predicts whether something is **True or False**.
- Instead of fitting a line, it fits an **S-shaped logistic function**.
- Ex- the curve tells whether a mouse is obese or not based on its weight. The curve depicts the probability of mouse being obese. $P(\text{obese}) > 0.5 \Rightarrow \text{mouse is obese}$.

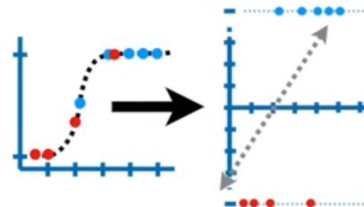


Logistic Regression (Methodology)

Exponentiate both sides...

$$\log\left(\frac{p}{1-p}\right) = \log(\text{odds})$$

$$\frac{p}{1-p} = e^{\log(\text{odds})}$$



Multiply both sides by $(1 - p)$...

$$p = (1 - p)e^{\log(\text{odds})}$$

Multiply $(1 - p)$ and $e^{\log(\text{odds})}$...

$$p = e^{\log(\text{odds})} - pe^{\log(\text{odds})}$$

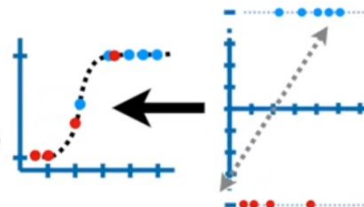
Add $pe^{\log(\text{odds})}$ to both sides... $p + pe^{\log(\text{odds})} = e^{\log(\text{odds})}$

Pull p out...

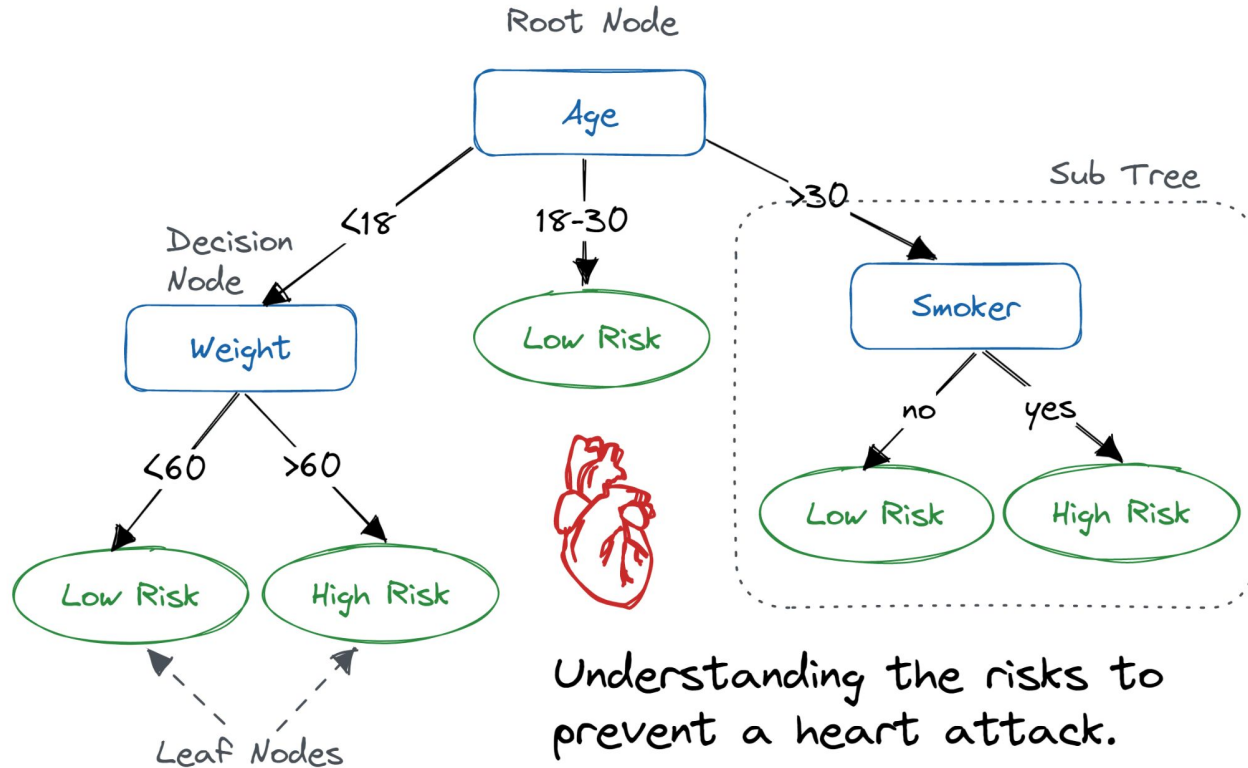
$$p(1 + e^{\log(\text{odds})}) = e^{\log(\text{odds})}$$

Divide both sides by $(1 + e^{\log(\text{odds})})$...

$$p = \frac{e^{\log(\text{odds})}}{1 + e^{\log(\text{odds})}}$$



Decision Trees



Decision Trees (Train and Inference)

GenDecTree(Sample S, Features F)

Steps:

1. **If** *stopping_condition(S, F) = true* **then**
 - a. *Leaf* = *createNode()*
 - b. *leafLabel* = *classify(s)*
 - c. **return** *leaf*
2. *root* = *createNode()*
3. *root.test_condition* = *findBestSpilt(S, F)*
4. $V = \{v \mid v \text{ a possible outcome of } \text{root.test_condition}\}$
5. **For each** value $v \in V$:
 - a. $S_v = \{s \mid \text{root.test_condition}(s) = v \text{ and } s \in S\}$;
 - b. *Child* = *TreeGrowth(S_v, F)*;
 - c. Add child as descent of root and label the edge {root $\rightarrow$ child} as v
6. **return** *root*

Input:

T *//Decision Tree*
 D *//Input Database*

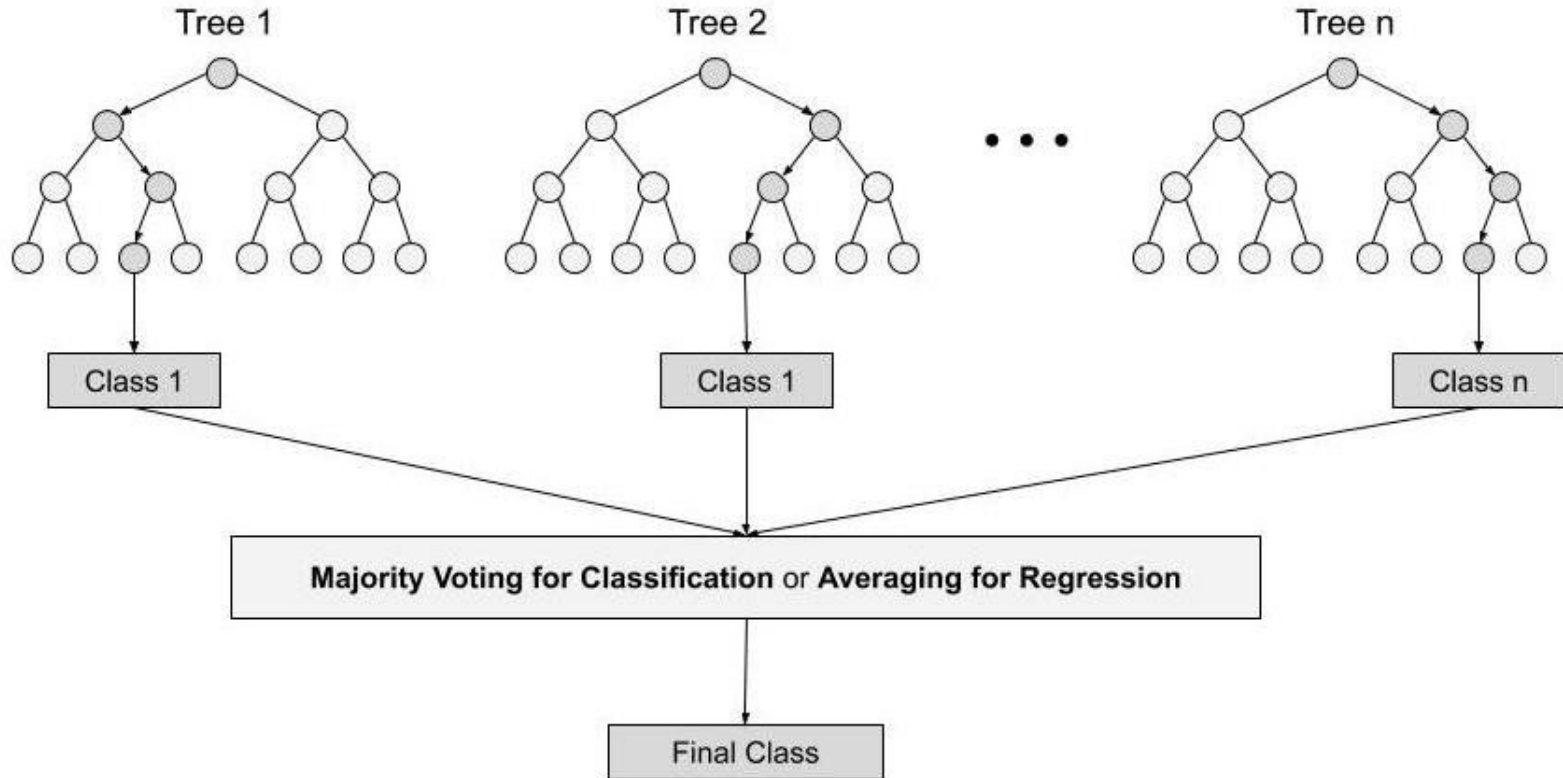
Output:

M *//Model Prediction*

DTProc Algorithm:

//Illustrates Prediction Technique using DT
for each $t \in D$ **do**
 n = root node of T ;
 while n not leaf node **do**
 Obtain answer to question on n applied t ;
 Identify arc from t which contains correct answer;
 n = node at end of this arc;
 Make prediction for t based on labeling of n ;

Random Forest (Ensemble of Decision Trees)



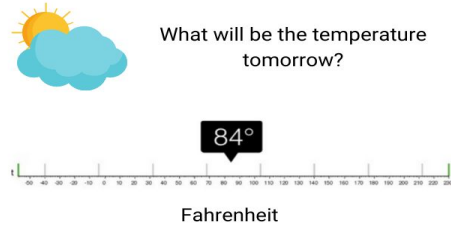
Scikit-Learn

What is Scikit-learn

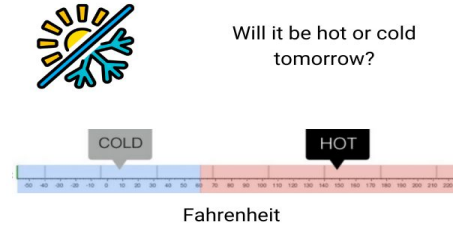
- Toolkit for data science and machine learning
- Built on NumPy, SciPy and Matplotlib.
- Open source with commercial usability

Tasks

Regression



Classification



- Regression
 - Predicting attribute value associated with an object
- Classification
 - Identifying the class to which an input instance belongs to:

Label Encoder

- Encode target labels with value between 0 and $n_classes-1$.
- This transformer should be used to encode target values, i.e., y , and not the input X .



The diagram illustrates the function of a Label Encoder. It shows a transformation from a categorical variable to a numerical one. On the left, a table with the header 'Height' contains the categories 'Tall', 'Medium', and 'Short'. A blue arrow points to the right, where a second table with the same header 'Height' shows the corresponding numerical encodings: 'Tall' is 0, 'Medium' is 1, and 'Short' is 2.

Height
Tall
Medium
Short

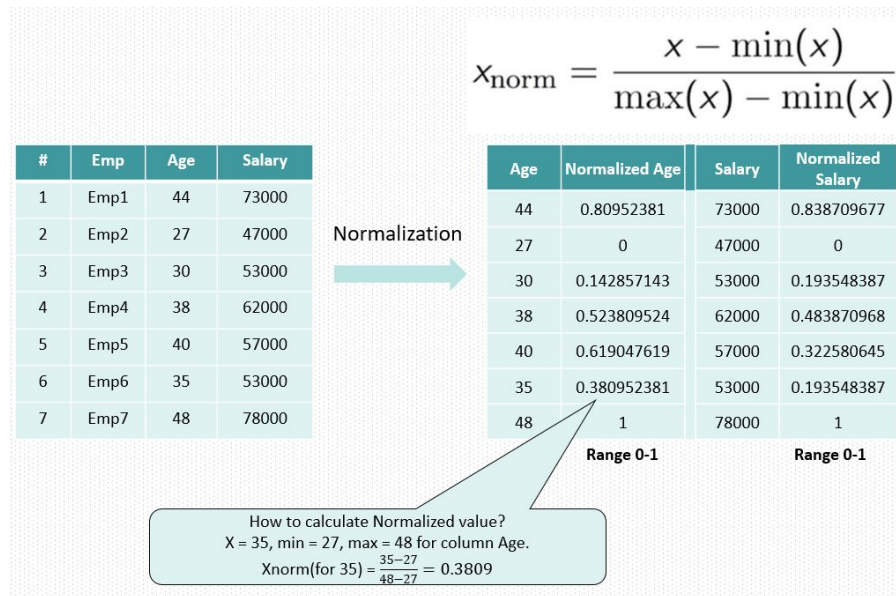
Height
0
1
2

Standard Scalar

- Standardize features by removing the mean and scaling to unit variance.

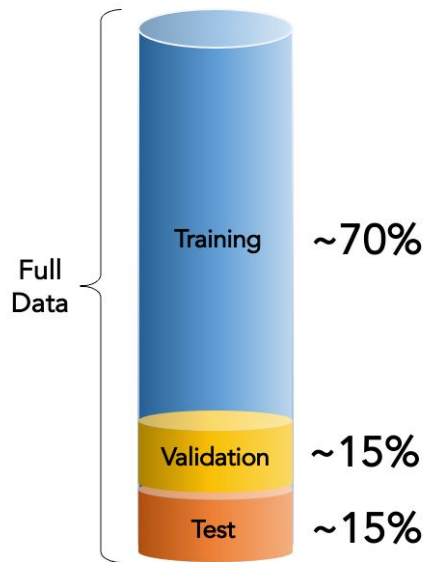
- The standard score of a sample x is calculated as:

- $z = (x - \mu) / \sigma$



Train Test Split

- The train-test split procedure is used to estimate the performance of machine learning algorithms when they are used for making predictions on data not used to train the model.
- Split arrays or matrices into random train and test subsets.



Implementation

**Thank You
&
Any Questions?**